Recover the Design

Author: Author

Reviewer: Reviewer (review pending)

Group number: withheld

Date: 2026-10-21

Abstract

This project recovers a fence layout from connector degrees. We connect neighboring connectors in each row and column, then use degree rules and backtracking to choose the fences. The C++17 program passes 935 local tests, including checks under AddressSanitizer and UndefinedBehaviorSanitizer. The search is complete, but its worst-case running time is exponential.

Contents

1 Introduction **3**

2 Data Structure / Algorithm Specification **3**

2.1 Candidate Edges 3

2.2 Propagation and Backtracking 3

2.3 Correctness Argument 4

3 Testing Results **4**

3.1 Environment and Method 4

3.2 Correctness Tests 5

3.3 Timing Results 6

4 Analysis and Comments **6**

4.1 Time Complexity 6

4.2 Space Complexity 6

4.3 Limitations and Improvements 7

5 Conclusion **7**

6 Author List **7**

7 Declaration **7**

8 Signatures **7**

9 Source Code Appendix **7**

1 Introduction

According to the PTA problem Recover the Design, each input contains one $n \times m$ grid, where $1 < n, m \leq 50$. A value from 1 to 4 gives a connector's required degree; zero means no connector. Fences are horizontal or vertical. They can turn or cross only at connectors.

For each connector, output `row col up down left right` in row order, from left to right. Coordinates start at 1; direction flags are 0 or 1. Separate numbers with one space, with no leading or trailing spaces. Print `No Solution` if no layout exists. Any valid layout is accepted. Separate components and cycles are allowed; an empty grid produces no output.

2 Data Structure / Algorithm Specification

2.1 Candidate Edges

A vertex represents a connector. A candidate edge joins two consecutive connectors in the same row or column. Each vertex has at most four edges. A fence through several connectors is represented by several edges.

Let $x_e \in \{0, 1\}$ show whether edge e is selected. For a vertex v with required degree k_v, require $\sum_{e \ni v} x_e = k_v$. If two edges e and f cross at an empty cell, require $x_e + x_f \leq 1$.

Row and column scans build the edges. Each edge stores its endpoints and their direction indices. Two grids record the horizontal and vertical edge through each empty cell. If both are present, add the pair to the conflict lists.

2.2 Propagation and Backtracking

Each edge is unknown, selected or excluded. At a vertex, let s count selected edges and u count unknown edges. Apply these rules:

1. If $s > k_v$ or $s + u < k_v$, reject the branch.
2. If $s = k_v$, exclude all remaining edges at this vertex.
3. If $s + u = k_v$, select all remaining edges at this vertex.
4. For each selected edge, exclude every edge that crosses it at an empty cell.

Repeat until no edge changes. If unknown edges remain, choose a vertex with the fewest unknown edges and branch on one of them. Try selecting the edge first, then excluding it. A list records changed edges; after failure, undo changes back to the saved position.

Stop when all edges are assigned. If both choices fail at every branch, return `No Solution`.

2.3 Correctness Argument

Split any valid fence at each connector it meets. Each segment joins consecutive connectors in a row or column, so every valid design uses only candidate edges.

The degree rules reject a branch only when it has too many selected edges or too few possible edges. They fix an edge only when the required degree forces that choice. Excluding a crossing edge is also necessary. Thus the rules do not remove any valid completion.

Each unknown edge has two possible values. The search tries both and restores the parent state after a failed branch. Each recursive call fixes at least one new edge, so the search terminates. By induction on the number of unknown edges, it finds a completion if one exists.

At success, all edges are assigned and all degree and crossing checks pass. Each selected edge sets matching flags at its two endpoints. Vertices are printed in input order. The output is therefore a valid design; if both choices fail in every branch, no solution exists.

3 Testing Results

3.1 Environment and Method

Tests ran on 2026-10-21 using WSL2 (Linux 6.6.87.2), an AMD Ryzen 7 H 255 CPU, GCC 13.3.0 and Python 3.13.13. The release build uses `-std=c++17 -O2 -Wall -Wextra -Wpedantic`. A separate build uses ASan and UBSan; commands are in `README.md`.

The checker follows each printed direction to the next connector. It checks position, order, spacing, degree, matching endpoint flags and crossings. For small grids, a separate checker tries every candidate-edge subset to decide whether a solution exists. Three invalid outputs test rejection of wrong degrees, unmatched endpoints and crossings.

3.2 Correctness Tests

Test case	Count	Expected / actual result
Supplied sample	1	Valid design; PASS
All 2 by 2 grids with entries 0 to 4	625	Match exhaustive check; PASS
Random 3 by 3 grids	200	Match exhaustive check; PASS
Crossing, junction, multiple solutions and straight-through cases	4	Expected outcomes; PASS
Generated valid 4 by 4 designs	100	Valid designs; PASS
Structured 50 by 50 grids	5	Expected outcomes; PASS

Both builds passed all 935 cases and the three checker rejection tests. No sanitizer error was reported. Random tests use seed 17.

An instrumented rerun found two valid 4 by 4 cases that each rejected one branch and recovered after one rollback, undoing 5 and 6 edge assignments. They are saved as `tests/rollback-a.in` and `tests/rollback-b.in`; run `scripts/check_rollback.py` to reproduce the counts. These are cases from the existing suite, not additional random trials. Failed searches across several levels or both choices of one branch remain untested.

3.3 Timing Results

All inputs are 50 by 50. Each has one warmup and ten measured runs. Times are in milliseconds and include process startup, input/output, solving and Python validation. The table gives the median and minimum–maximum range; it does not measure solver time alone.

Input	Median (ms)	Range (ms)
Empty grid	1.868	1.392 to 2.131
Degree 1 everywhere	41.276	38.732 to 42.976
All neighboring edges required	8.283	7.785 to 10.294
Degree 1 at both ends of each row	2.292	1.932 to 2.749
Degree 4 everywhere (no solution)	2.520	2.085 to 2.909

The degree-1 grid needs branching. The full-edge grid is forced by degree rules, and the degree-4 grid fails at a corner. These cases do not cover the hardest searches. The README gives reproduction commands; local timing files are not part of the submission.

4 Analysis and Comments

4.1 Time Complexity

Let V count vertices, E candidate edges and X crossing pairs. Each vertex has at most four edges, so $E \leq 2V$. Each empty cell has at most one crossing pair, so $X \leq nm$. Candidate interiors do not overlap within one row or column. Building and marking them therefore takes $O(nm+V+E+X)$ time.

One rule scan takes $O(V+E+X)$. Each scan that changes the state fixes at least one unknown edge, so propagation needs at most $E+1$ scans per search node. A branch fixes an edge; search depth is at most E, with at most $2^{E+1}-1$ nodes. Combining these bounds gives $O(nm+2^E(E+1)(V+E+X))$ worst-case time. This bound is derived from the code; the tests do not establish a better bound.

4.2 Space Complexity

The grids, edges and conflict lists use $O(nm+V+E+X)$ total space. The change list and recursion stack use $O(E)$ auxiliary space. Each branch changes the shared state instead of copying the graph.

4.3 Limitations and Improvements

1. Search time is exponential in the worst case. PTA acceptance has not been checked, and the supplied statement does not include time or memory limits.
2. Propagation scans the whole graph. A queue of affected vertices and edges could reduce repeated work.
3. The large tests are structured cases. Add harder sparse and nearly impossible cases to study search cost.

5 Conclusion

Degree rules reduce the choices, and backtracking tries the remaining ones. The method finds a valid layout if one exists, but its worst-case search time remains exponential.

6 Author List

Author: code, tests, report and responses to review comments.

Reviewer: independent reproduction and approval (pending).

7 Declaration

This report is a draft. The group declaration remains to be confirmed before submission.

8 Signatures

Author: pending confirmation.

Reviewer: review and signature pending.

9 Source Code Appendix

The solver is in `code/main.cpp`, the checker in `scripts/check.py`, and the sample in `tests/sample1.in`. See `README.md` for build and test commands.